Spring Boot Application Report

Student–Course Management System

GitHub Repository

Project Link:
https://github.com/Advita9/springboot-student-course-controller

1. Introduction

This project is a web-based application developed using **Spring Boot** to manage information for two entities: **Students** and **Courses**. The system supports core operations such as **Create, Read, and Update**, and demonstrates relational data handling using **JPA and SQL JOIN queries**.

The application follows a **layered architecture** consisting of Controller, Service, Repository, and Model layers, ensuring separation of concerns and scalability.

2. Entity Relationship Design

The system models a **Many-to-Many relationship** between Students and Courses.

- A student can enroll in multiple courses
- A course can have multiple students

This relationship is implemented using a **join table**: `student_course`.

Entity Diagram (Conceptual)

Student <---> Student_Course <---> Course

Entity Classes

- `Student.java`

```java
package com.example.demo.model;

import jakarta.persistence.*;
import java.util.List;

@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    public Long getId() {
    return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    private String name;
    private String email;

    @ManyToMany
    @JoinTable(
        name = "student_course",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private List<Course> courses;
}
```

- `Course.java`

```java
package com.example.demo.model;

import jakarta.persistence.*;
import java.util.List;

@Entity
public class Course {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
    private int credits;

    @ManyToMany(mappedBy = "courses")
    private List<Student> students;
}
```

Key Annotations Used

- `@Entity` - Defines database tables
- `@Id`, `@GeneratedValue` - Primary key
- `@ManyToMany` - Defines relationship
- `@JoinTable` - Configures join table

3. Implementation Details

3.1 Create Operation

Description

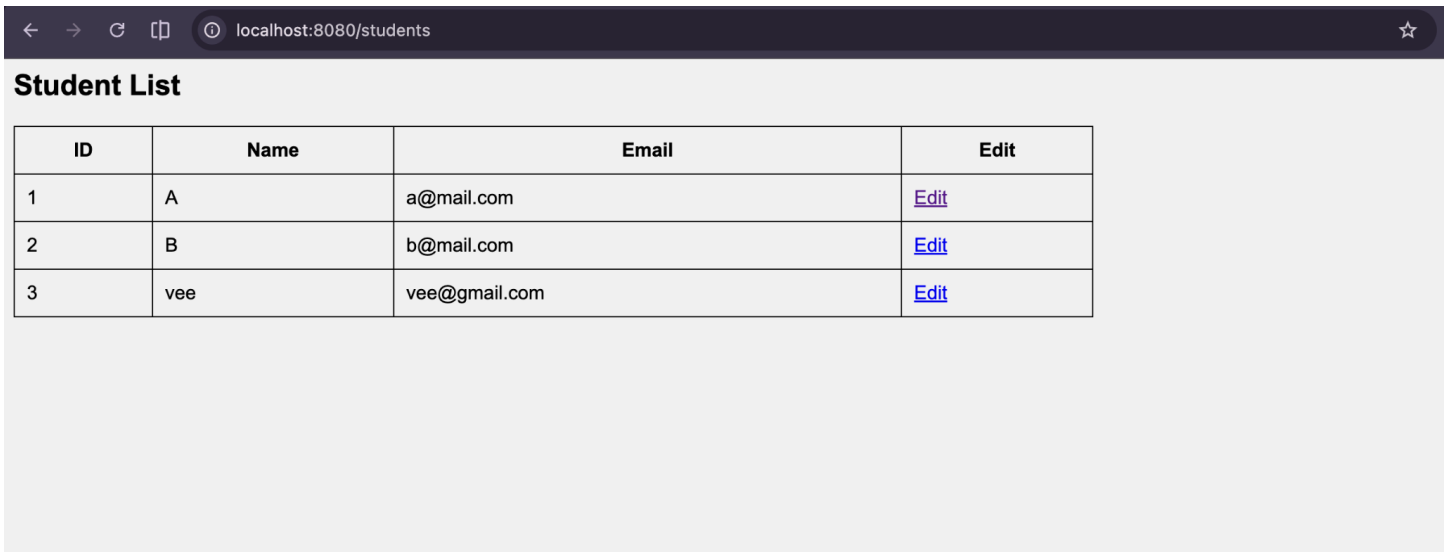
Users can add a new student using a JSP form. The data is submitted via a POST request and saved using the service and repository layers.

Controller Logic

```java
@PostMapping("/save")
public String save(@ModelAttribute Student student, Model model) {
  try {
    service.save(student);
  } catch (Exception e) {
    model.addAttribute("error", "Error saving student!");
    return "student-form";
  }
  return "redirect:/students";
}
```

- Add Student Form (`/add`)

- After submitting form (new student visible)



3.2 Read Operation

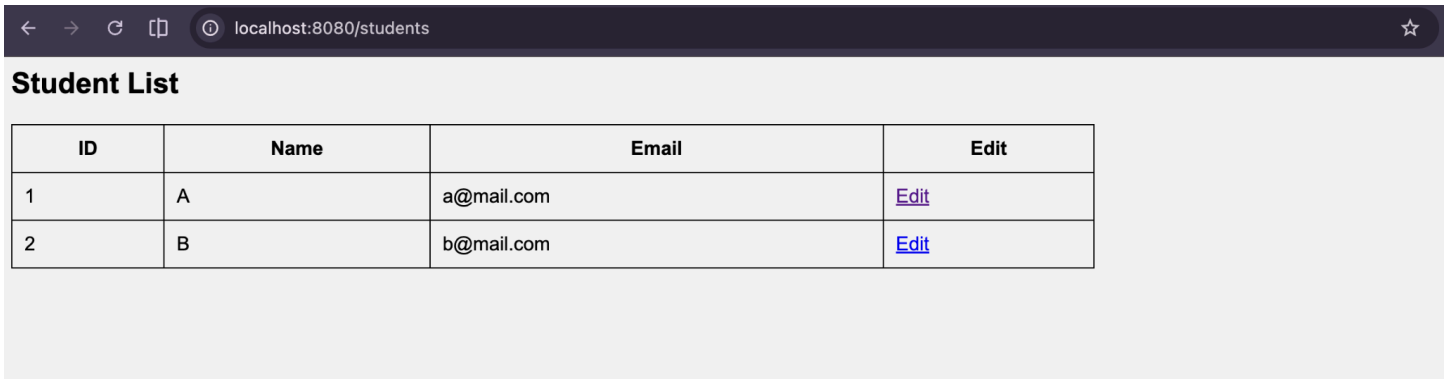
Description

Displays all students in a tabular format using JSP and JSTL.

Controller Logic

```
@GetMapping("/students")
public String list(Model model) {
  model.addAttribute("students", service.getAll());
  return "student-list";
}
```

- Student list page (`/students`)



3.3 Update Operation

Description

Allows updating existing student data through a pre-filled form.

Controller Logic

```
@PostMapping("/update")
public String update(@ModelAttribute Student student) {
  service.save(student);
  return "redirect:/students";
}
```

- Edit page (`/edit/{id}`)

- After update

3.4 Custom JOIN Query (Important Feature)

Description

A custom JPQL query is used to fetch relational data between Students and Courses.

Repository Method

```
@Query("SELECT s.name, c.title FROM Student s JOIN s.courses c")
List<Object[]> getStudentCourseDetails();
```

Controller Mapping

```
@GetMapping("/report")
public String report(Model model) {
  model.addAttribute("data", service.getJoinData());
  return "report";
}
```

- `/report` page showing student-course mapping

Student - Course Mapping

Student: A | Course: Math
Student: A | Course: Physics
Student: B | Course: Math

4. Database Design & Initialization

- Database: **H2 (in-memory)**
- Tables created automatically using JPA
- Sample data inserted using `data.sql`
- H2 Console (`/h2-console`)
- Tables: STUDENT, COURSE, STUDENT_COURSE

5. Frontend (JSP + CSS)

- JSP used for rendering UI
- JSTL used for dynamic data binding
- CSS applied for basic styling

6. Testing

A basic unit test was implemented using **JUnit** to validate service layer functionality.

Example Test

```
@Test
void testSaveStudent() {
  Student s = new Student();
  s.setName("Test");
  s.setEmail("test@mail.com");

  assertNotNull(service.save(s));
}
```

- Test file

```java
package com.example.demo;

import com.example.demo.model.Student;
import com.example.demo.service.StudentService;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;

import static org.junit.jupiter.api.Assertions.*;

@SpringBootTest
class StudentServiceTest {

    @Autowired
    StudentService service;

    @Test
    void testSaveStudent() {
        Student s = new Student();
        s.setName("Test");
        s.setEmail("test@mail.com");

        Student saved = service.save(s);

        assertNotNull(saved);
    }
}
```

7. Challenges Faced & Solutions

1. H2 Table Not Found Error

- **Issue:** `Table STUDENT not found`
- **Cause:** `data.sql` executed before table creation
- **Solution:**
  - Added:

    spring.jpa.defer-datasource-initialization=true

2. Primary Key Constraint Error

- **Issue:** Duplicate ID error while inserting data
- **Cause:** Manual IDs conflicting with auto-generated IDs
- **Solution:**
  - Removed manual IDs from `data.sql`
  - Allowed JPA to auto-generate IDs

3. JOIN Query Returning Empty Results

- **Issue:** No data in `/report` page
- **Cause:** Join table not populated
- **Solution:**
  - Added entries to `student_course` table

4. Missing Imports / Compilation Errors

- **Issue:** `cannot find symbol` errors
- **Cause:** Missing imports and incorrect packages
- **Solution:**
  - Added proper `jakarta.persistence` and Spring imports

8. Conclusion

The project successfully demonstrates:

- Implementation of **Spring Boot MVC architecture**
- Handling of **entity relationships using JPA**
- Execution of **CRUD operations**
- Use of **custom JOIN queries**
- Integration of **JSP-based frontend**

The system is modular, scalable, and fulfills all assignment requirements.